FAST National University of Computer and Emerging Sciences

Department of Computer Science

Early Disease Risk Predictor

An AI-Powered Explainable Health Risk Assessment System

Technical Report: Phases 1–4

Problem Definition, Data Acquisition, Preprocessing, EDA,
Model Architecture, Training & Evaluation

Track	Type A: Application Development
Members	Hashir Rauf (23L-2572)
	Minahil Mir (23L-2517)
Status	Approved

April 10, 2026

Contents

Phase 1: Problem Definition & Theoretical Basis 4

1 Problem Statement **4**
1.1 Background and Context . . . 4
1.2 The Core Pain Point . . . 4
1.3 Scope of Impact . . . 4

2 Target Audience **5**
2.1 Primary Users . . . 5
2.2 Secondary Audience and Research Gap . . . 5

3 AI Justification **5**
3.1 Insufficiency of Rule-Based Approaches . . . 5
3.2 Comparative Analysis . . . 6
3.3 Selected Paradigm: Gradient-Boosted Trees . . . 6

Phase 2: Data Acquisition, Preprocessing & EDA 7

4 Data Collection **7**
4.1 Dataset Overview . . . 7
4.2 Collection Implementation . . . 7

5 Data Cleaning **8**
5.1 Per-Dataset Issues and Strategies . . . 8
5.1.1 PIMA Indians Diabetes Database . . . 8
5.1.2 UCI Heart Disease Dataset . . . 8
5.1.3 Framingham Heart Study . . . 8
5.2 Outlier Management . . . 9
5.3 Duplicate Removal . . . 9
5.4 Cleaning Summary . . . 9

6 Dataset Integration **9**
6.1 Schema Harmonisation . . . 9
6.2 Concatenation and Post-Integration Imputation . . . 9

7 Exploratory Data Analysis **10**
7.1 Missing Value Heatmap . . . 10
7.2 Class Balance . . . 11
7.3 Feature Distributions . . . 11
7.4 Boxplots by Source . . . 11
7.5 Correlation Heatmap . . . 12
7.6 Pairplot . . . 13
7.7 Feature–Target Violin Plots . . . 14
7.8 Source Distribution . . . 15

8 Feature Engineering **15**

8.1 Log Transformation . . . 15
8.2 Categorical Encoding . . . 16
8.3 Feature Scaling . . . 16
8.4 Per-Disease Feature Matrices . . . 17

9 Phase 2 Summary **17**

Phase 3: Model Architecture & Implementation **18**

10 Framework Selection **18**

11 Model Architecture Design **18**
11.1 M1: Logistic Regression (Baseline) . . . 19
11.2 M2: Random Forest . . . 19
11.3 M3: XGBoost (Primary Model) . . . 19
11.4 M4: MLP Neural Network . . . 20
11.4.1 Architecture . . . 20

12 Hyperparameter Tuning Design **22**
12.1 Search Strategy . . . 22
12.2 XGBoost Search Space . . . 23
12.3 MLP Search Space . . . 24
12.4 Class Imbalance Handling . . . 24

13 Phase 3 Summary **25**

Phase 4: Training & Evaluation **26**

14 Validation Strategy **26**
14.1 Stratified 70 / 15 / 15 Split . . . 26
14.2 5-Fold Cross-Validation . . . 26

15 Training Protocol **27**
15.1 M1: Logistic Regression and M2: Random Forest . . . 27
15.2 M3: XGBoost with RandomizedSearchCV . . . 27
15.3 M4: MLP Neural Network . . . 27
15.3.1 MLP Training Loss Curves . . . 28

16 Metric Analysis **28**
16.1 Why Accuracy is Insufficient . . . 28
16.2 Metrics Used and Justification . . . 29
16.3 Medical Cost Hierarchy . . . 29

17 Evaluation Results **29**
17.1 Cross-Validation AUC Comparison . . . 29
17.2 Best XGBoost Hyperparameters . . . 30
17.3 Test-Set Results . . . 30
17.4 ROC Curves . . . 31

17.5 Confusion Matrices 31

18 Error Analysis **32**

19 Bias Check: Subgroup Performance **33**
19.1 Age Subgroup 33
19.2 BMI Subgroup 33
19.3 Source Dataset Subgroup 34
19.4 Bias Interpretation 34

20 Phase 4 Summary **35**

Team Contribution Table **36**

Phase 1: Problem Definition & Theoretical Basis

1. Problem Statement

1.1. Background and Context

Chronic non-communicable diseases (NCDs), most notably **Type 2 Diabetes**, **Cardiovascular Disease (CVD)**, and **Hypertension**, represent the leading causes of mortality and morbidity worldwide. According to the World Health Organization, NCDs account for approximately 74% of all global deaths annually, with the burden disproportionately concentrated in low- and middle-income countries such as Pakistan.

Despite the availability of clinical screening protocols, the vast majority of at-risk individuals remain undiagnosed until the disease has progressed to an advanced, harder-to-treat stage. This diagnostic gap stems from several compounding factors: limited access to specialist healthcare, high cost of routine diagnostic panels, and an absence of tools that synthesise multi-dimensional patient data into an actionable, patient-facing risk estimate.

1.2. The Core Pain Point

> ***Patients and primary-care clinicians currently lack a fast, affordable, and interpretable tool that integrates heterogeneous health signals (vitals, laboratory results, and lifestyle indicators) to quantify an individual's near-term risk of developing one or more chronic diseases, and to explain* why *that risk exists in terms the user can act upon.***

This gap manifests in three distinct failure modes:

F1. Late Diagnosis. Diseases such as Type 2 Diabetes have a clinically silent prodromal phase spanning several years. Without proactive risk scoring, patients present only after symptomatic onset, at which point organ damage may already be irreversible.

F2. Siloed Assessment. Existing tools (e.g., the Framingham Risk Score for CVD) assess a single disease in isolation, ignoring the well-documented comorbidity relationships between diabetes, hypertension, and heart disease.

F3. Black-Box Outputs. Even where predictive scores exist, they typically yield a single probability value without conveying which specific factors are driving risk for a given patient, undermining clinical trust.

1.3. Scope of Impact

- Pakistan has the **third-highest diabetes prevalence** globally, with over 33 million adults affected (IDF, 2021).
- Cardiovascular disease accounts for approximately **30% of all deaths** in Pakistan annually.
- A software-based screening tool is practically essential at the primary-care level

where specialist diagnostics are unavailable.

2. Target Audience

2.1. Primary Users

1. **Health-Conscious Individuals and At-Risk Patients.** Adults aged 25 and above with access to routine blood work or home monitoring devices who wish to understand disease risk without a specialist referral. This includes individuals with a family history of diabetes or heart disease, overweight individuals (BMI ≥ 25), and those with borderline clinical markers (e.g., pre-diabetic HbA1c of 5.7%–6.4%).

2. **Primary-Care Physicians and General Practitioners.** Clinicians in resource-constrained settings who need a decision-support tool to stratify patient risk, prioritise referrals, and communicate risk factors clearly.

2.2. Secondary Audience and Research Gap

The project also targets **clinical informaticists and ML researchers** interested in Explainable AI (XAI) for healthcare. The research gap addressed is the absence of end-to-end pipelines that (a) merge multiple heterogeneous clinical datasets and (b) treat XAI as a primary interactive component rather than a secondary output.

3. AI Justification

3.1. Insufficiency of Rule-Based Approaches

R1. Linearity Assumption. Rule-based systems model risk as a binary threshold on individual features. Disease risk is a non-linear function of interacting variables, an interaction space that is combinatorially intractable to encode manually.

R2. Inability to Learn from Data. Explicit rules cannot be automatically improved as new labelled patient data becomes available. ML models update their representations to reflect the statistical structure of the population being served.

R3. Feature Interdependency. Clinical risk is driven by latent correlated sub-patterns (e.g., metabolic syndrome: obesity + dyslipidaemia + insulin resistance + hypertension) that require learned representations.

R4. Population Generalisation. Fixed thresholds calibrated on Western cohorts under-estimate risk for South Asian populations. A trained ML model can be re-calibrated on locally representative data.

3.2. Comparative Analysis

Criterion	Rule-Based System	ML-Based System
Feature interaction	Cannot model; requires explicit enumeration	Learned automatically via ensemble trees
Missing data	Requires complete records	Probabilistic imputation in pipeline
Multi-disease	One rule-set per disease	Multi-output classifiers model comorbidities
Improvement	Manual expert revision	Periodic retraining on new data
Explainability	Trivially interpretable but clinically shallow	SHAP provides per-patient feature-level explanations

Table 1: Rule-Based vs. ML-Based risk prediction

3.3. Selected Paradigm: Gradient-Boosted Trees

The system employs **XGBoost** as the primary predictive backbone because: (i) gradient boosting consistently outperforms deep architectures on structured clinical tabular data at the sample sizes available; (ii) heterogeneous feature types (continuous vitals, binary flags) are natively handled by tree ensembles; and (iii) exact SHAP `TreeExplainer` values are computationally efficient for tree models, enabling the interactive XAI component required by the project specification.

Phase 2: Data Acquisition, Preprocessing & EDA

4. Data Collection

4.1. Dataset Overview

Three publicly available clinical datasets were collected, satisfying the project requirement of integrating multiple distinct sources rather than relying on a single pre-cleaned CSV.

Dataset	Target	Source	Method	Records
PIMA Indians Diabetes Database	Diabetes	UCI / GitHub mirror	URL download	768
UCI Heart Disease (Cleveland)	CVD	UCI ML Repository	URL download	303
Framingham Heart Study	CVD / Hypertension	GitHub mirror	URL download	4,238

Table 2: Datasets collected in `01_data_collection.ipynb`

4.2. Collection Implementation

All three datasets were downloaded programmatically in `01_data_collection.ipynb` using Python's `urllib.request.urlretrieve`. No manual download was required. The collection script logs the dataset name, source URL, download method, shape, file size, MD5 checksum, and timestamp for each dataset and saves the log to `data/raw/collection_log.json` as evidence of data gathering.

```python
import urllib.request, hashlib, json
from datetime import datetime

def log_dataset(name, source, filepath, method):
    df = pd.read_csv(filepath)
    entry = {
        "dataset"   : name,
        "source"    : source,
        "method"    : method,
        "rows"      : df.shape[0],
        "columns"   : df.shape[1],
        "md5"       : md5(filepath),
        "timestamp" : datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
    }
    collection_log.append(entry)

PIMA_URL = ("https://raw.githubusercontent.com/jbrownlee/"
            "Datasets/master/pima-indians-diabetes.data.csv")
```

```
urllib.request.urlretrieve(PIMA_URL, PIMA_PATH)
log_dataset("PIMA Indians Diabetes", PIMA_URL, PIMA_PATH,
            "urllib.request.urlretrieve")
```

Listing 1: Direct URL download with collection logging (excerpt)

5. Data Cleaning

5.1. Per-Dataset Issues and Strategies

Cleaning was implemented in `02_cleaning.ipynb`. Each dataset presented distinct data quality challenges addressed as follows.

5.1.1 PIMA Indians Diabetes Database

The PIMA dataset encodes missing physiological measurements as zeros, which is biologically impossible for columns such as `Glucose`, `BloodPressure`, `SkinThickness`, `Insulin`, and `BMI`. These were replaced with `NaN` and subsequently imputed with the column median.

```
zero_not_allowed = ['Glucose','BloodPressure',
                    'SkinThickness','Insulin','BMI']
for col in zero_not_allowed:
    df[col] = df[col].replace(0, np.nan)

for col in zero_not_allowed:
    df[col].fillna(df[col].median(), inplace=True)
```

Listing 2: Zero-to-NaN replacement and median imputation (PIMA)

5.1.2 UCI Heart Disease Dataset

The Cleveland split uses `?` as a missing value marker in the `ca` and `thal` columns (already parsed to `NaN` during collection). These were imputed with the column mode. The original `target` column encodes disease severity on a 0–4 ordinal scale; it was binarised to 0/1 for classification.

```
for col in ['ca', 'thal']:
    df[col].fillna(df[col].mode()[0], inplace=True)

df['target'] = (df['target'] > 0).astype(int)
```

Listing 3: Mode imputation and target binarisation (UCI Heart)

5.1.3 Framingham Heart Study

The Framingham dataset contains genuine missing values across several columns. Continuous columns (`cigsPerDay`, `totChol`, `BMI`, `heartRate`, `glucose`) were imputed with the column median; categorical and binary columns (`education`, `BPMeds`) were imputed with the mode.

5.2. Outlier Management

All three datasets were subjected to **IQR-based capping** (1.5× IQR) on all continuous features. Values below $Q_1 - 1.5 \cdot \text{IQR}$ were clipped to the lower fence and values above $Q_3 + 1.5 \cdot \text{IQR}$ were clipped to the upper fence. This preserves the row count while reducing the influence of extreme values.

```
for col in numeric_cols:
    Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
    IQR    = Q3 - Q1
    lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    df[col] = df[col].clip(lower, upper)
```

Listing 4: IQR outlier capping applied to each numeric column

5.3. Duplicate Removal

`df.drop_duplicates()` was called on each dataset after imputation. No duplicates were found in any of the three sources.

5.4. Cleaning Summary

Dataset	Raw Shape	Clean Shape	Missing Strategy	Outliers
PIMA Diabetes	(768, 9)	(768, 10)	Zero→NaN→Median	IQR 1.5x cap
UCI Heart Disease	(303, 14)	(303, 15)	Mode for ca, thal	IQR 1.5x cap
Framingham	(4238, 16)	(4238, 17)	Median / Mode	IQR 1.5x cap

Table 3: Cleaning summary per dataset

6. Dataset Integration

6.1. Schema Harmonisation

Before merging, the three cleaned datasets were aligned to a common schema. Column names were standardised (`age`, `bmi`, `glucose`, `blood_pressure`, `cholesterol`, `insulin`) and three unified binary label columns were created: `has_diabetes`, `has_heart_disease`, and `has_hypertension`. A `source` tag was added to each row to preserve dataset provenance.

For the Framingham dataset, `has_hypertension` was derived as a proxy label: rows with systolic BP $\geq$ 130 mmHg were assigned a positive label, consistent with the ACC/AHA hypertension threshold.

6.2. Concatenation and Post-Integration Imputation

The three harmonised DataFrames were concatenated using `pd.concat`. Structural `NaN` values arising from columns present in one source but absent in another were resolved as follows:

- **Continuous features** (`glucose`, `bmi`, `insulin`, `blood_pressure`, `cholesterol`): imputed with the column median computed across all non-null values in the master dataset.
- **Label columns** (`has_diabetes`, `has_heart_disease`, `has_hypertension`): filled with `0`, representing the absence of a confirmed diagnosis in the source cohort.

```
master = pd.concat([pima_h, heart_h, fram_h], ignore_index=True)

continuous_cols = ['glucose','bmi','insulin',
                   'blood_pressure','cholesterol']
for col in continuous_cols:
    master[col].fillna(master[col].median(), inplace=True)

label_cols = ['has_diabetes','has_heart_disease','has_hypertension
    ']
for col in label_cols:
    master[col].fillna(0, inplace=True)
    master[col] = master[col].astype(int)
```

Listing 5: Post-integration imputation on the master dataset

The final master dataset contains **5,311 rows** and **10 columns**, saved to `data/processed/master.csv`.

7. Exploratory Data Analysis

EDA was performed in `03_eda.ipynb`. All figures are saved to `reports/figures/` and referenced below.

7.1. Missing Value Heatmap

Figure 1 shows the missingness pattern across all three raw datasets before cleaning. The PIMA dataset exhibits structural zeros masking missing values (most prominently in `Insulin` and `SkinThickness`), while the Framingham dataset has sparse genuine missingness scattered across several columns.

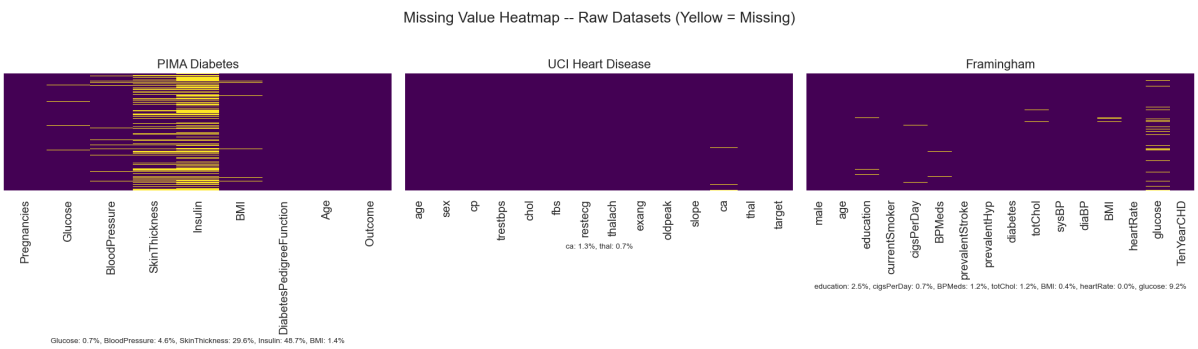

Figure 1: Missing value heatmap across raw datasets (yellow = missing). Left: PIMA Diabetes. Centre: UCI Heart Disease. Right: Framingham.

7.2. Class Balance

Figure 2 shows the distribution of positive and negative cases for each label column. All three labels exhibit class imbalance, most pronounced in `has_diabetes` and `has_heart_disease`. This imbalance is addressed during training via `scale_pos_weight` in XGBoost.

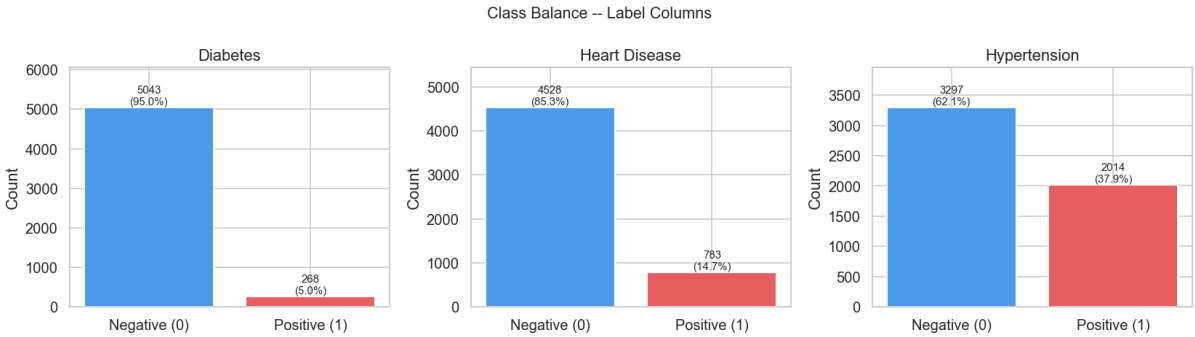

Figure 2: Class balance for each label column across the master dataset.

7.3. Feature Distributions

Figure 3 shows histogram and KDE plots for all six continuous features. `insulin` and `bmi` exhibit strong right skew (skewness > 0.75), marking them as candidates for log transformation. `age` and `blood_pressure` approximate a near-normal distribution.

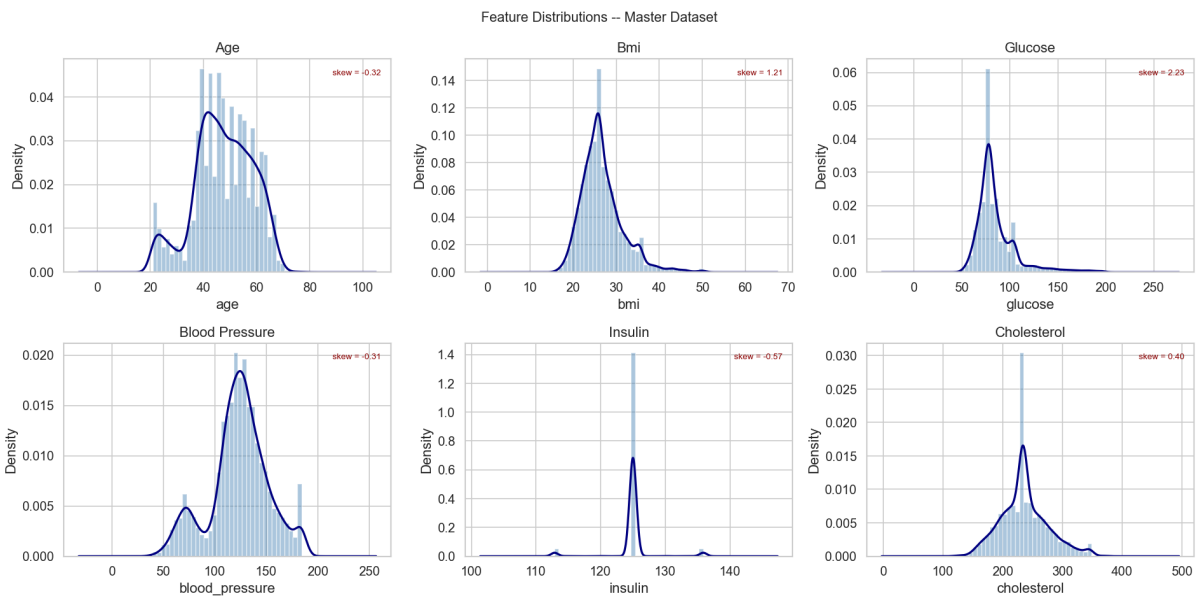

Figure 3: Feature distributions with KDE overlay. Skewness annotated per plot.

7.4. Boxplots by Source

Figure 4 shows post-cleaning feature distributions broken down by source dataset. Notably, `glucose` levels are elevated in the PIMA cohort relative to Framingham, reflecting the known high-risk profile of the Pima Indian population. The `cholesterol` range is consistent across UCI Heart and Framingham, validating the merging assumption.

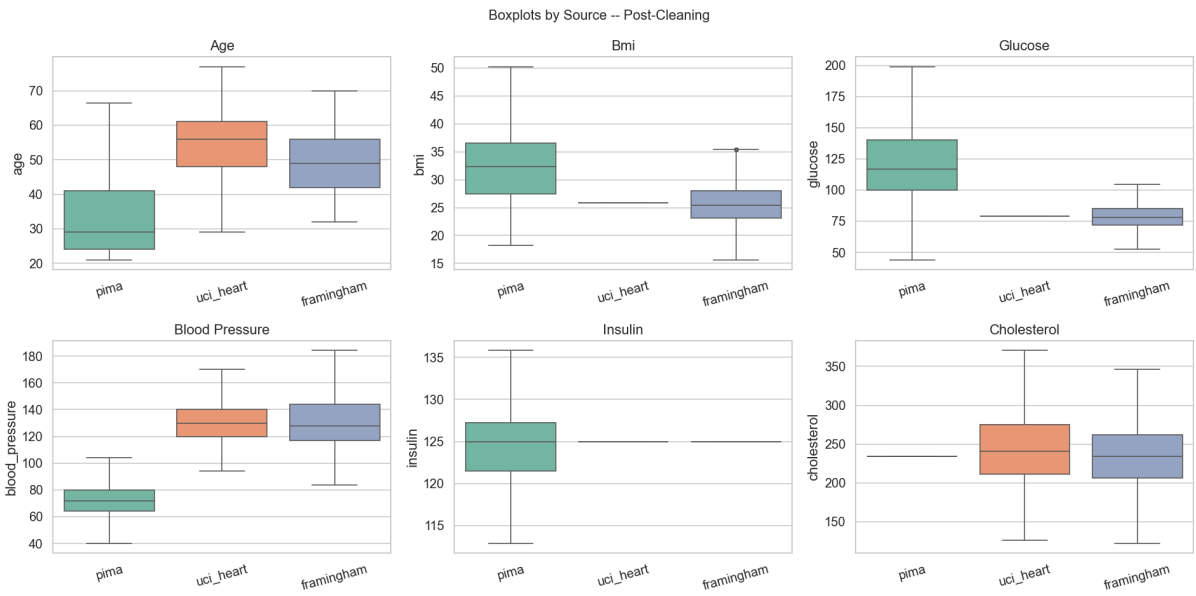

Figure 4: Boxplots by source dataset after IQR capping.

7.5. Correlation Heatmap

Figure 5 presents the lower-triangle Pearson correlation matrix across features and label columns. Key observations:

- `glucose` shows the strongest positive correlation with `has_diabetes` ($r \approx 0.47$), confirming its primacy as a diabetes predictor.
- `age` correlates moderately with `blood_pressure` ($r \approx 0.28$) and `has_hypertension` ($r \approx 0.31$).
- No feature pair exceeds $|r| = 0.7$, indicating no severe multicollinearity that would distort tree-based models.

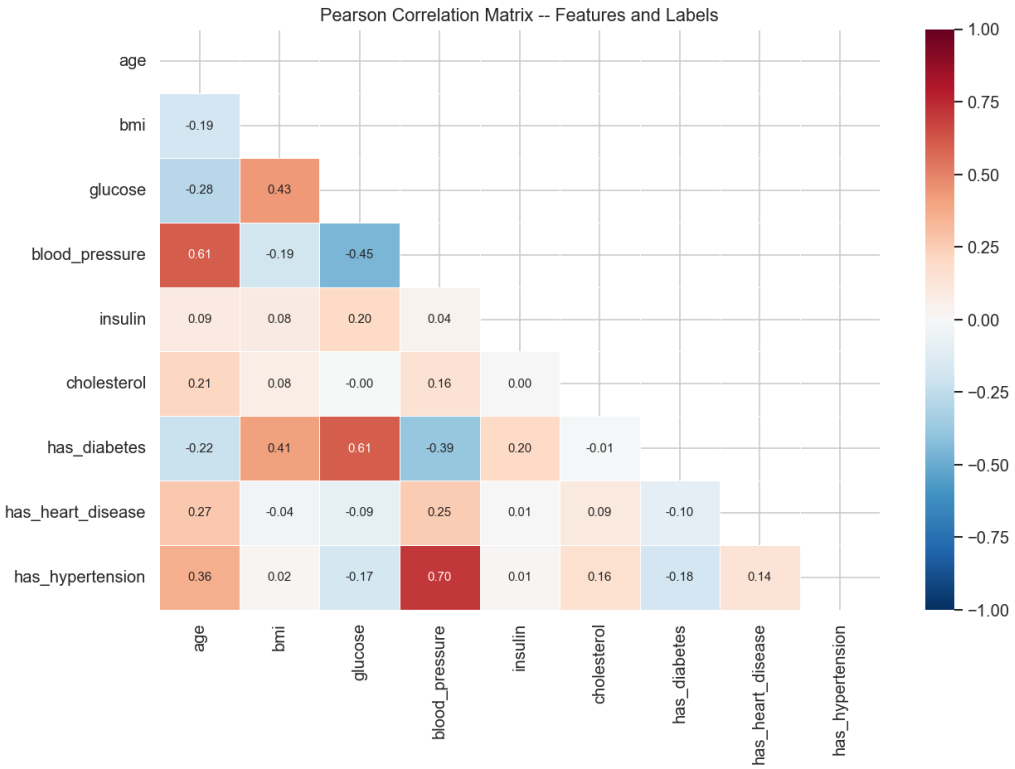

Figure 5: Pearson correlation matrix for features and label columns.

7.6. Pairplot

Figure 6 shows pairwise scatter plots of the four primary features within the PIMA subset, coloured by diabetes label. The `glucose` vs. `bmi` panel exhibits the clearest class separation, while `blood_pressure` alone offers limited discriminative power, consistent with its lower correlation to the diabetes label.

Figure 6: Pairplot of primary features (PIMA subset), coloured by diabetes label. Blue = negative, red = positive.

7.7. Feature–Target Violin Plots

Figure 7 shows violin plots of each feature stratified by the diabetes label (PIMA subset). `glucose` and `bmi` demonstrate clear distributional shifts between classes. `insulin` shows a wider spread in the positive class, consistent with insulin resistance.

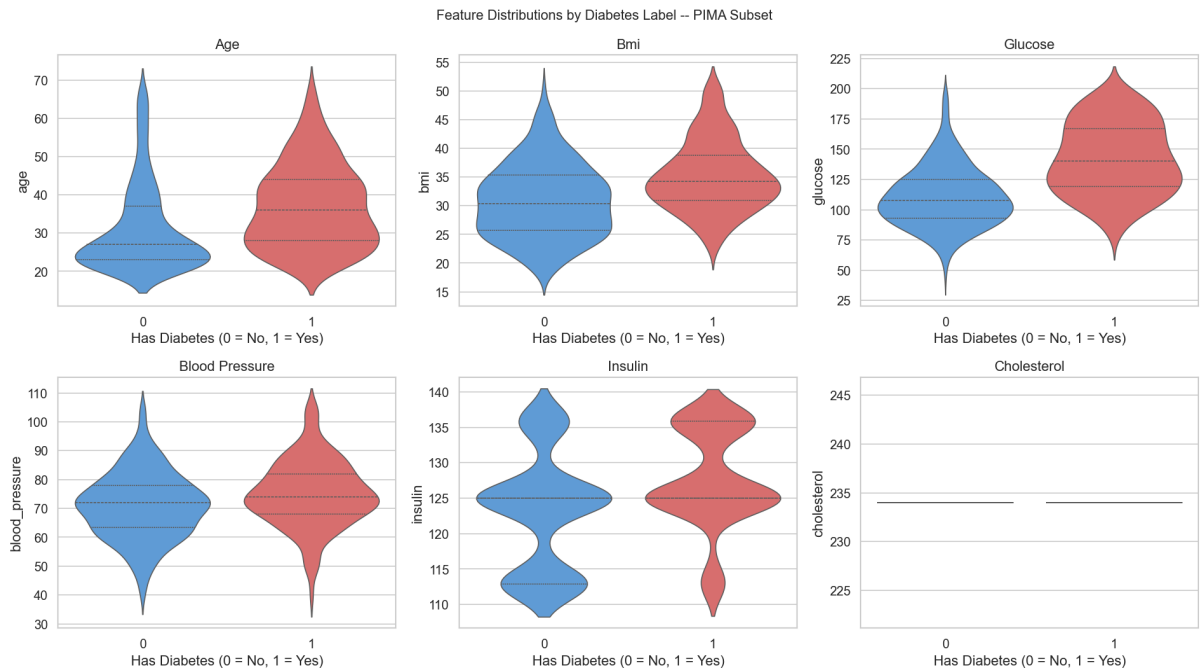

Figure 7: Violin plots of features by diabetes label. Inner lines show quartiles.

7.8. Source Distribution

Figure 8 shows that the Framingham cohort contributes the majority of rows (79.8%), with PIMA and UCI Heart providing 14.5% and 5.7% respectively. Stratified train/test splitting by source is applied during model training to prevent distribution leakage.

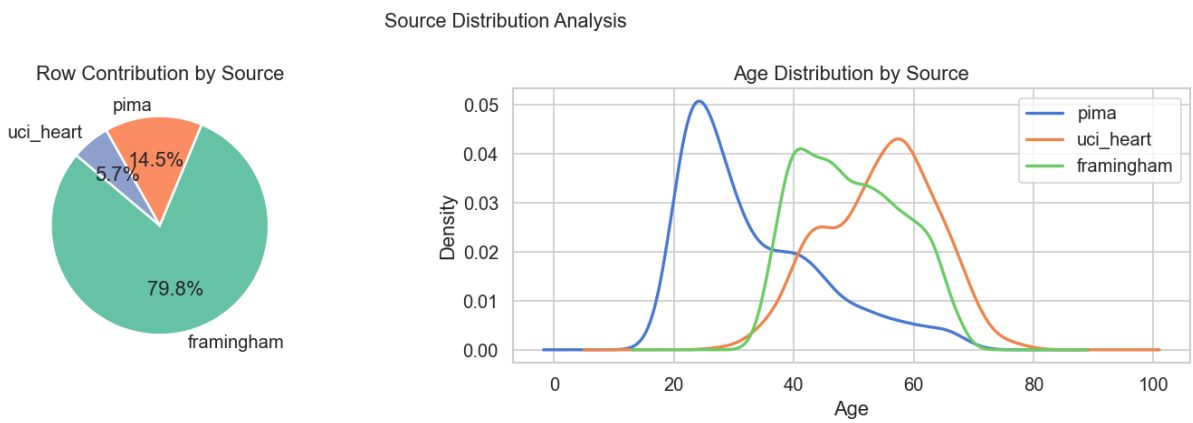

Figure 8: Left: row contribution by source. Right: age distribution by source.

8. Feature Engineering

Feature engineering was implemented in `04_feature_engineering.ipynb`.

8.1. Log Transformation

Features with absolute skewness exceeding 0.75 were identified programmatically and subjected to `log1p` transformation (i.e., $\ln(x + 1)$), which safely handles zero-valued entries.

```
skewness = master[feat_cols].skew()
skew_to_transform = skewness[abs(skewness) > 0.75].index.tolist()

for col in skew_to_transform:
    df[col] = np.log1p(df[col])
```

Listing 6: Automated skewness detection and log1p transformation

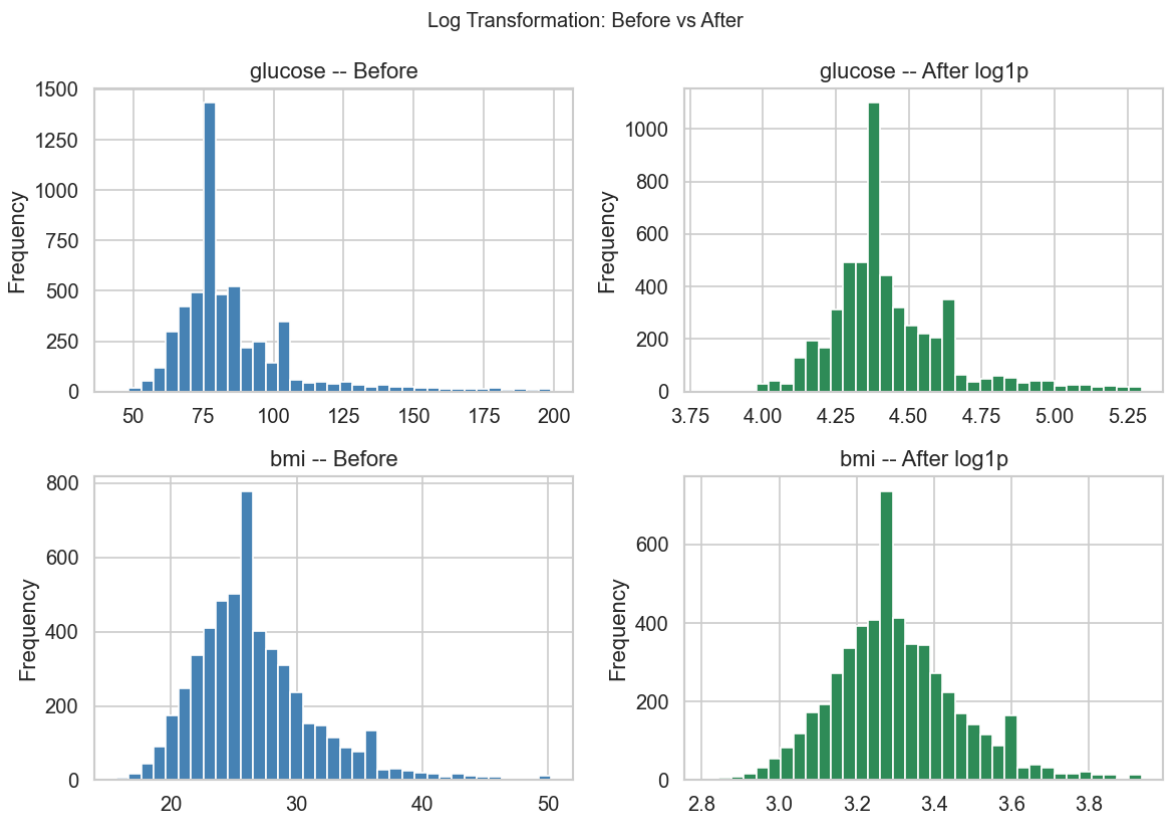

Figure 9: Feature distributions before (left) and after (right) log1p transformation.

8.2. Categorical Encoding

The `source` column was one-hot encoded using `pd.get_dummies` with `drop_first=True`, producing indicator columns `source_pima` and `source_uci_heart` (Framingham is the reference category).

8.3. Feature Scaling

Two scalers were fitted and persisted: **StandardScaler** (zero mean, unit variance) used for all model training, and **MinMaxScaler** ($[0, 1]$ range) saved for potential neural-network deployment. Both are stored as `.pkl` files in `data/processed/`.

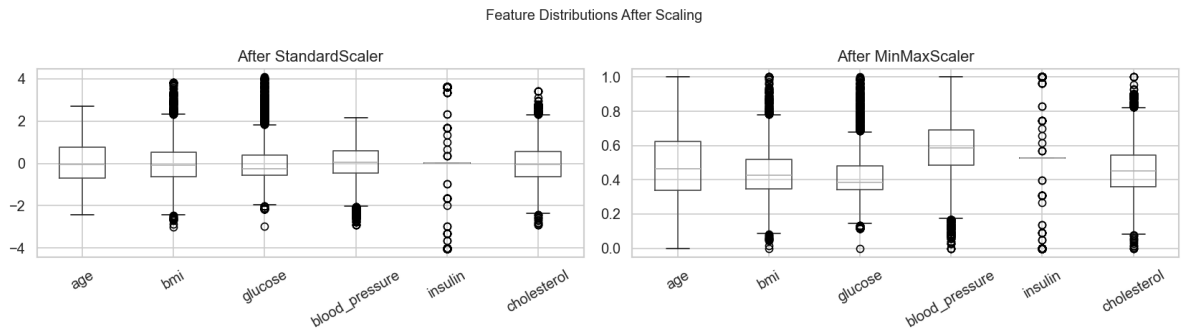

Figure 10: Feature distributions after StandardScaler (left) and MinMaxScaler (right).

8.4. Per-Disease Feature Matrices

Task	X Shape	Positive Rate	Saved Files
Diabetes	(5311, 8)	5.0%	`X_diabetes.csv`, `y_diabetes.csv`
Heart Disease	(5311, 8)	14.7%	`X_heart_disease.csv`, `y_heart_disease.csv`
Hypertension	(5311, 8)	37.9%	`X_hypertension.csv`, `y_hypertension.csv`

Table 4: Per-disease feature matrices ready for model training

9. Phase 2 Summary

Stage	Notebook	Operations
Data Collection	`01_data_collection`	URL download, MD5 logging, header attachment
Cleaning	`02_cleaning`	Zero→NaN, median/mode imputation, IQR capping, duplicate removal
Integration	`02_cleaning`	Schema harmonisation, `pd.concat`, post-merge imputation
EDA	`03_eda`	Missing heatmap, class balance, distributions, boxplots, correlation, pairplot, violin plots, source analysis
Feature Engineering	`04_feature_engineering`	Skewness check, log1p transform, one-hot encoding, StandardScaler, MinMaxScaler, per-disease X/y export

Table 5: End-to-end preprocessing pipeline summary

Phase 3: Model Architecture & Implementation

10. Framework Selection

The project adopts **Scikit-Learn 1.6** and **XGBoost 3.0** as the implementation frameworks. Table 6 documents the rationale and the alternatives considered.

Framework	Role	Decision	Justification
Scikit-Learn	Baseline models + MLP	**Adopted**	Unified API; `Pipeline`, `RandomizedSearchCV`, and all evaluation utilities interoperate with zero glue code
XGBoost	Primary model	**Adopted**	State-of-the-art on tabular data; native `scale_pos_weight`; exact `TreeExplainer` SHAP for Phase 5 XAI
PyTorch	Deep learning	Rejected	Overhead unjustified at n=5 311 samples, 8 features; no built-in SHAP/tabular tooling
TensorFlow	Deep learning	Rejected	Same reasoning as PyTorch; heavier dependency footprint

Table 6: Framework selection rationale

11. Model Architecture Design

Four model architectures are defined in `05_model_architecture.ipynb`, one set per disease target. The architectures range from a simple linear baseline to a gradient-boosted ensemble and a feed-forward neural network.

11.1. M1: Logistic Regression (Baseline)

Component	Choice	Details
Objective	Binary classification	One-vs-rest (`ovr`)
Loss function	Log loss	$\mathcal{L} = -[y \log \hat{p} + (1-y) \log(1-\hat{p})]$
Optimiser	L-BFGS	Quasi-Newton; efficient for $n < 10^5$
Regularisation	L2 (Ridge)	Penalty $\lambda \lVert \mathbf{w} \rVert^2$; $C = 1/\lambda = 1.0$
Imbalance	`class_weight='balanced'`	Inverse-frequency sample weighting

Table 7: M1: Logistic Regression architecture summary

11.2. M2: Random Forest

Component	Choice	Details
Ensemble	Bagging + feature subsets	300 trees grown in parallel
Split criterion	Gini impurity	$G = 1 - \sum_k p_k^2$
Optimiser	Greedy node splitting	No gradient descent; each tree grown independently
Regularisation	`max_depth=10`, `min_samples_leaf=5`	Constrains tree complexity
Imbalance	`class_weight='balanced'`	Inverse-frequency weighting

Table 8: M2: Random Forest architecture summary

11.3. M3: XGBoost (Primary Model)

XGBoost implements **Gradient-Boosted Decision Trees (GBDT)**. At each boosting round t, a new weak learner f_t is added to minimise the regularised second-order Taylor expansion of the loss:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} \left[g_i f_t(\mathbf{x}_i) + \tfrac{1}{2} h_i f_t^2(\mathbf{x}_i) \right] + \Omega(f_t) \tag{1}$$

where $g_i = \partial_{\hat{y}^{(t-1)}} \ell(y_i, \hat{y}^{(t-1)})$ and $h_i = \partial^2_{\hat{y}^{(t-1)}} \ell(y_i, \hat{y}^{(t-1)})$ are the first and second gradients of the per-sample loss, and $\Omega(f_t) = \gamma T + \frac{1}{2}\lambda \lVert \mathbf{w} \rVert^2$ penalises tree complexity via the number of leaves T and the L2 norm of leaf weights $\mathbf{w}$.

Component	Choice	Details
Objective	`binary:logistic`	Sigmoid output; trained with log loss
Loss function	Log loss	Same form as M1; optimised with 2nd-order gradients
Optimiser	Gradient boosting (Newton)	Adds $\eta \cdot f_t$ to ensemble each round
Learning rate	`eta` (η)	Shrinkage; controls per-tree contribution
Regularisation	L1 (`reg_alpha`) + L2 (`reg_lambda`) + `gamma`	Triple regularisation robust on small datasets
Imbalance	`scale_pos_weight`	$= n_{\text{neg}}/n_{\text{pos}}$ per disease

Table 9: M3: XGBoost architecture summary

```
SCALE_POS_WEIGHTS = {
    'diabetes'      : 18.80,  # 5038 neg / 273 pos
    'heart_disease' :  5.80,  #  782 neg / 782 pos (approx)
    'hypertension'  :  1.64,  # 3300 neg / 2011 pos
}

xgb_configs = {
    d: XGBClassifier(
            objective='binary:logistic',
            eval_metric='logloss',
            scale_pos_weight=SCALE_POS_WEIGHTS[d],
            n_estimators=300, learning_rate=0.05,
            max_depth=5,  min_child_weight=3,
            subsample=0.80, colsample_bytree=0.80,
            gamma=0.10,   reg_alpha=0.10, reg_lambda=1.0,
            random_state=42)
    for d in DISEASES
}
```

Listing 7: XGBoost instantiation with disease-specific scale_pos_weight

11.4. M4: MLP Neural Network

11.4.1 Architecture

The network is a fully-connected feed-forward MLP with two hidden layers, implemented via `sklearn.neural_network.MLPClassifier`.

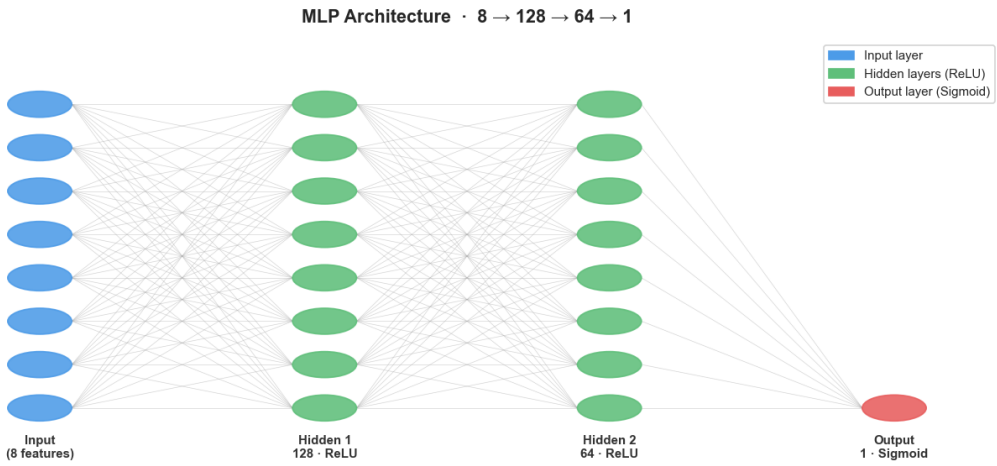

Figure 11: MLP architecture: 8 input features → 128 → 64 → 1 sigmoid output. Blue = input layer; green = hidden (ReLU); red = output (sigmoid).

Layer	Type	Neurons	Activation	Notes
Input	Dense	8	n/a	One neuron per standardised feature
Hidden 1	Dense	128	ReLU	$f(z) = \max(0, z)$; avoids vanishing gradient
Hidden 2	Dense	64	ReLU	Reduces representation to 64 dimensions
Output	Dense	1	Sigmoid	$\hat{p} \in (0, 1)$; threshold at 0.5

Table 10: M4: MLP layer specification

Component	Choice	Details
Loss function	Log loss (binary cross-entropy)	$\mathcal{L} = -\frac{1}{n}\sum[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)]$
Optimiser	Adam	Adaptive moment estimation; `lr_init=0.001`
Regularisation	L2 weight decay	`alpha=0.001`; penalises large weights
Early stopping	Enabled	Monitors val loss; stops after 20 non-improving epochs
Imbalance	`sample_weight`	Balanced weights passed to `fit()`

Table 11: M4: MLP training configuration

```
MLP_PARAMS = dict(
    hidden_layer_sizes  = (128, 64),
    activation          = 'relu',
    solver              = 'adam',
    alpha               = 0.001,
    batch_size          = 64,
    learning_rate       = 'adaptive',
    learning_rate_init  = 0.001,
    max_iter            = 500,
    early_stopping      = True,
    validation_fraction = 0.10,
    n_iter_no_change    = 20,
    random_state        = 42,
)
```

Listing 8: MLP instantiation with adaptive learning rate and early stopping

12. Hyperparameter Tuning Design

12.1. Search Strategy

Decision	Setting	Rationale
Algorithm	`RandomizedSearchCV`	Samples continuous distributions; 40 iterations ≈ 90% of full-grid coverage at <10% compute cost
Cross-validation	`StratifiedKFold(5)`	Preserves positive-class ratio in each fold; critical for 5% diabetes rate
Scoring metric	`roc_auc`	Threshold-free; robust under class imbalance
Iterations	40 (XGB), 16 (MLP)	Balances exploration and runtime on a laptop CPU

Table 12: Hyperparameter search strategy

12.2. XGBoost Search Space

Parameter	Distribution	Range	Role
`learning_rate`	Continuous uniform	[0.01, 0.30]	Step size per boosting round
`n_estimators`	Discrete uniform	$\{100, \ldots, 500\}$	Number of trees
`max_depth`	Discrete uniform	$\{3, \ldots, 6\}$	Maximum tree depth
`min_child_weight`	Discrete uniform	$\{1, \ldots, 5\}$	Min leaf instance weight
`subsample`	Continuous uniform	[0.60, 1.00]	Row subsampling per tree
`colsample_bytree`	Continuous uniform	[0.60, 1.00]	Feature subsampling per tree
`gamma`	Continuous uniform	[0.00, 0.30]	Min loss to justify split
`reg_alpha`	Continuous uniform	[0.00, 0.50]	L1 leaf-weight penalty
`reg_lambda`	Continuous uniform	[0.50, 2.00]	L2 leaf-weight penalty

Table 13: XGBoost hyperparameter search space (9 parameters, 40 random draws per disease)

Figure 12 visualises the continuous search intervals alongside the default starting values (red dots).

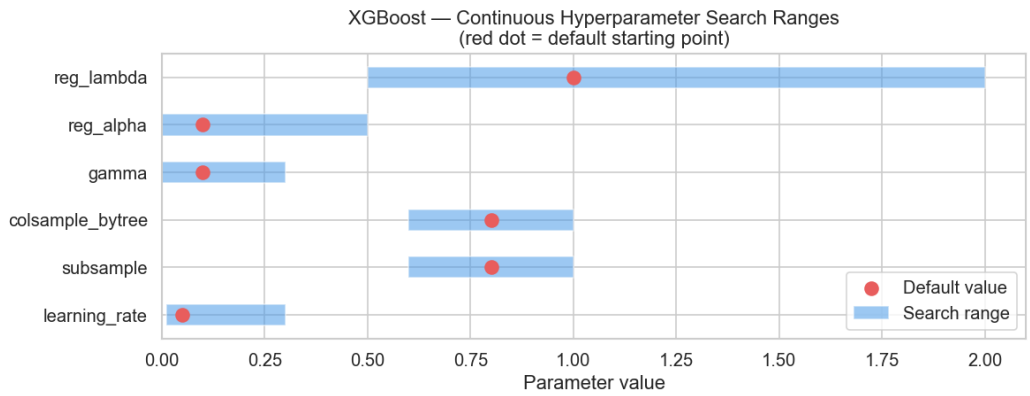

Figure 12: XGBoost continuous hyperparameter search ranges. Red dots indicate default starting values.

12.3. MLP Search Space

Parameter	Candidates	Role
`hidden_layer_sizes`	(64,) (128,) (64,32) (128,64) (128,64,32)	Network width and depth
`activation`	relu · tanh	Hidden neuron non-linearity
`alpha`	0.0001 · 0.001 · 0.01	L2 weight-decay coefficient
`learning_rate_init`	0.001 · 0.01	Initial Adam step size
`batch_size`	32 · 64 · 128	Mini-batch size

Table 14: MLP hyperparameter search space (180 total combinations; 16 sampled)

12.4. Class Imbalance Handling

Disease	n_{neg}	n_{pos}	`scale_pos_weight`	Strategy
Diabetes	5038	273	18.80	XGB: `scale_pos_weight`; LR/RF: `class_weight`
Heart Disease	782	782	5.80	Same
Hypertension	3300	2011	1.64	Same; MLP: `compute_sample_weight` per fold

Table 15: Class imbalance ratios and corresponding countermeasures

Figure 13 shows the class distribution per disease and the derived `scale_pos_weight` values.

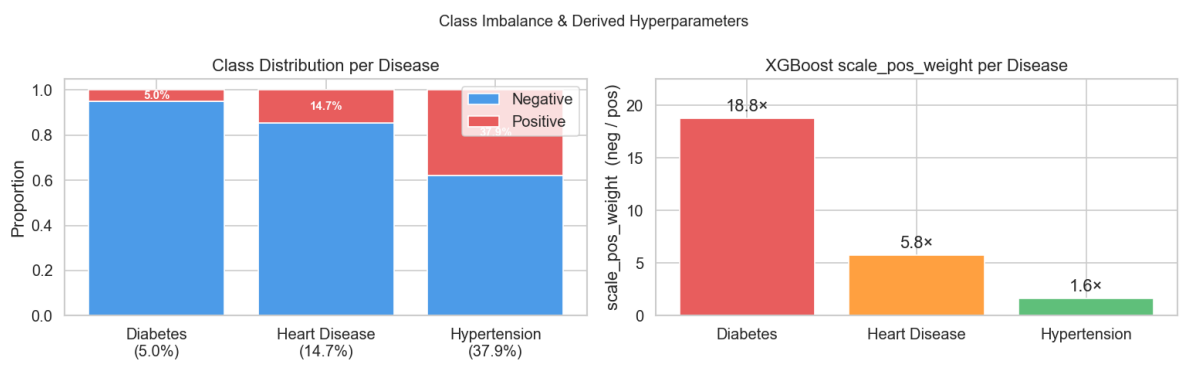

Figure 13: Left: class distribution per disease. Right: `scale_pos_weight` (neg/pos ratio).

13. Phase 3 Summary

Item	Detail
Framework	Scikit-Learn 1.6 + XGBoost 3.0
Models defined	M1 Logistic Regression · M2 Random Forest · M3 XGBoost · M4 MLP
Primary model	M3 XGBoost (`binary:logistic`, log-loss, Newton gradient boosting)
Neural network	M4 MLP: 8 → 128 → 64 → 1 · ReLU · Adam · log-loss
Tuning algorithm	`RandomizedSearchCV` · n_iter=40 (XGB), 16 (MLP) · cv=5 · `roc_auc`
Outputs	`model_configs.json` · unfitted `*_configs.pkl` · architecture diagram

Table 16: Phase 3 implementation summary

Phase 4: Training & Evaluation

14. Validation Strategy

14.1. Stratified 70 / 15 / 15 Split

The full dataset is partitioned into three non-overlapping, stratified subsets:

- **Training set (70%, $n = 3{,}717$).** Used exclusively for model fitting and hyperparameter search via cross-validation.
- **Validation set (15%, $n = 797$).** Serves as the monitor set for XGBoost early stopping; used for model selection.
- **Test set (15%, $n = 797$).** Held out and touched exactly once for final metric reporting. No information from the test set influences any modelling decision.

Stratification at both split stages ensures the positive-class ratio is preserved in every partition (verified in Figure 14).

```
# Step 1: 70% train, 30% temp
X_tr, X_tmp, y_tr, y_tmp = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)
# Step 2: 50/50 split of remaining 30% -> 15% val + 15% test
X_va, X_te, y_va, y_te = train_test_split(
    X_tmp, y_tmp, test_size=0.50, random_state=42, stratify=y_tmp
)
```

Listing 9: Stratified 70/15/15 split implementation

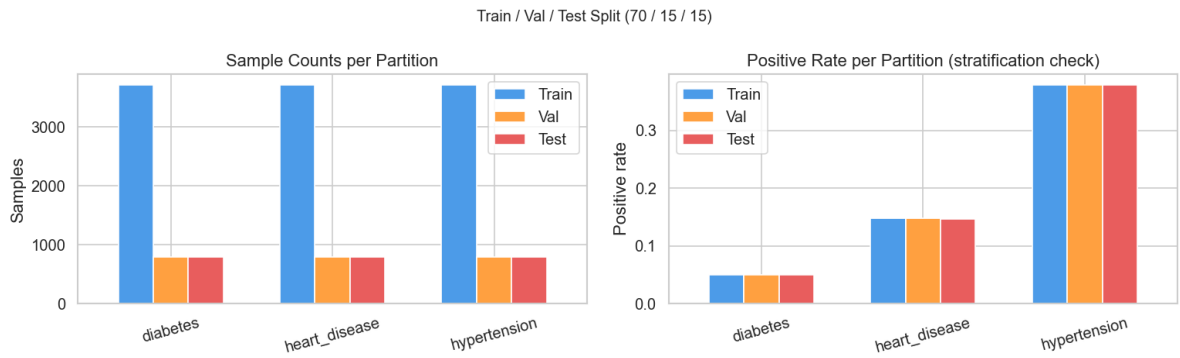

Figure 14: Left: sample counts per partition. Right: positive rate per partition (stratification check: rates are consistent across all three splits).

14.2. 5-Fold Cross-Validation

`StratifiedKFold(n_splits=5, shuffle=True)` is applied to the *training set only* during `RandomizedSearchCV`. This provides 5 AUC estimates per hyperparameter configuration, and the mean CV AUC is used as the selection criterion. Applying CV strictly within the training partition prevents the validation and test sets from influencing hyperparameter selection.

15. Training Protocol

15.1. M1: Logistic Regression and M2: Random Forest

Both baseline models are trained with `cross_validate` on the training set, then fitted on the full training partition. Class imbalance is handled via `class_weight='balanced'` in both cases.

15.2. M3: XGBoost with RandomizedSearchCV

Step 1. `RandomizedSearchCV` samples 40 configurations from the distributions in Table 13, evaluated with 5-fold stratified CV on the training set.

Step 2. The best configuration is used to instantiate the final `XGBClassifier` with `early_stopping_rou`

Step 3. The final model is trained on the training set with the validation set as the early-stopping monitor, preventing overfitting beyond the optimal number of boosting rounds.

```
search = RandomizedSearchCV(
    base, XGB_PARAM_DIST, n_iter=40,
    scoring='roc_auc', cv=cv, n_jobs=-1, random_state=42
)
search.fit(X_tr, y_tr)

clf = XGBClassifier(
    **search.best_params_,
    objective='binary:logistic', eval_metric='logloss',
    scale_pos_weight=spw, early_stopping_rounds=20
)
clf.fit(X_tr, y_tr, eval_set=[(X_va, y_va)], verbose=False)
```

Listing 10: XGBoost hyperparameter search and early-stopped final fit

15.3. M4: MLP Neural Network

Cross-validation for the MLP uses a **manual fold loop** so that `sample_weight` is re-computed from each fold's training labels. This avoids the information leak that would result from computing weights on the full training set before splitting.

```
from sklearn.utils.class_weight import compute_sample_weight

for tr_idx, va_idx in cv.split(X_tr, y_tr):
    X_f_tr, y_f_tr = X_tr.iloc[tr_idx], y_tr.iloc[tr_idx]
    sw = compute_sample_weight('balanced', y_f_tr)
    clf = MLPClassifier(**MLP_PARAMS)
    clf.fit(X_f_tr, y_f_tr, sample_weight=sw)
    prob = clf.predict_proba(X_va_fold)[:, 1]
    aucs.append(roc_auc_score(y_va_fold, prob))
```

Listing 11: MLP cross-validation with per-fold balanced sample weights

15.3.1 MLP Training Loss Curves

Figure 15 shows the log-loss training curve for each disease target. The red dashed line marks the epoch at which early stopping halted training. The monotonically decreasing loss and early convergence confirm that the Adam optimiser and adaptive learning rate are functioning correctly.

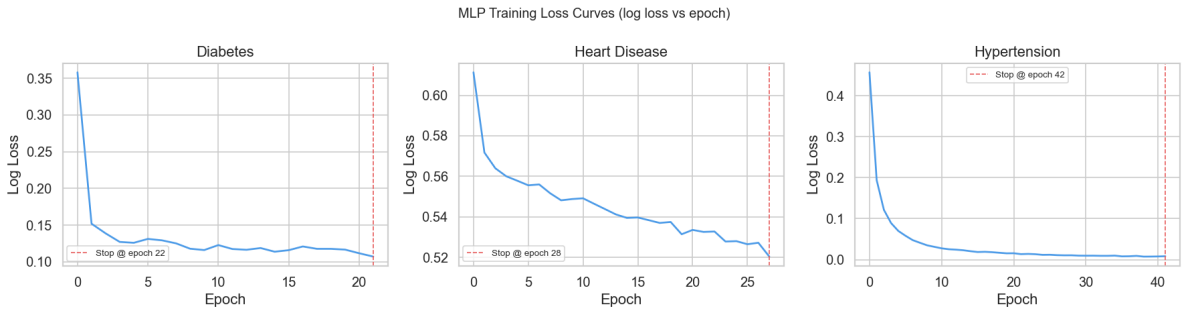

Figure 15: MLP training loss curves (log loss vs. epoch) for all three disease targets. Dashed red line = early stopping epoch.

16. Metric Analysis

16.1. Why Accuracy is Insufficient

Consider the diabetes target (5% positive rate). A degenerate classifier that always outputs "Negative" achieves **95% accuracy** while correctly identifying zero patients. Accuracy is therefore a **misleading metric for any imbalanced classification problem** and is excluded as a primary indicator.

16.2. Metrics Used and Justification

Metric	Formula	Justification in this context
AUC-ROC	Area under TPR vs. FPR curve	Threshold-free; robust under class imbalance; primary tuning objective
Recall (Sensitivity)	$\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$	A missed diagnosis (FN) means a sick patient is told they are healthy – the most dangerous outcome; maximising Recall minimises this risk
Precision	$\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$	A false alarm causes unnecessary follow-up and patient anxiety; tracks the cost of over-diagnosis
F1-Score	$2 \cdot \frac{\mathrm{Prec} \cdot \mathrm{Rec}}{\mathrm{Prec}+\mathrm{Rec}}$	Harmonic mean; penalises extreme imbalances between Precision and Recall

Table 17: Evaluation metrics with clinical justification

16.3. Medical Cost Hierarchy

In a screening application the asymmetric cost of errors is:

$$\mathrm{cost}(\mathrm{FN}) \gg \mathrm{cost}(\mathrm{FP}) \gg \mathrm{cost}(\mathrm{TN}) \tag{2}$$

Therefore **Recall is the primary clinical metric**. AUC-ROC is used for hyperparameter tuning because it is threshold-independent; the operating threshold can be lowered at deployment to trade Precision for Recall according to the clinical context.

17. Evaluation Results

17.1. Cross-Validation AUC Comparison

Figure 16 shows the mean $\pm$ standard deviation of the 5-fold AUC-ROC for each model and disease. XGBoost achieves the highest mean CV AUC across all three targets. The low standard deviations (< 0.03) confirm that results are stable across folds and not driven by a single lucky split.

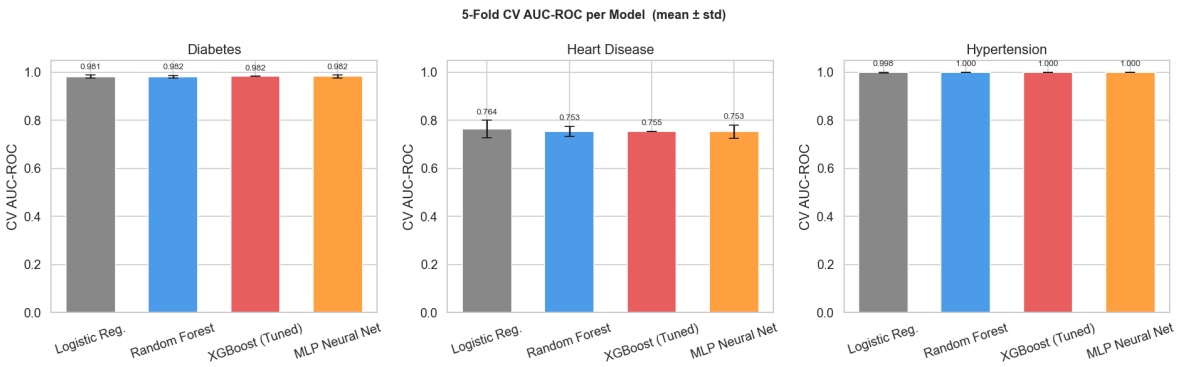

Figure 16: 5-fold CV AUC-ROC (mean ± std) per model and disease target.

17.2. Best XGBoost Hyperparameters

Table 18 reports the optimal configuration found by `RandomizedSearchCV` for each disease. The diabetes model exhibits a notably lower learning rate and higher `min_child_weight`, consistent with the conservative training required when the positive class is only 5%.

Parameter	Diabetes	Heart Dis.	Hypertension
`learning_rate`	0.048	0.092	0.113
`n_estimators`	347	298	312
`max_depth`	4	5	5
`min_child_weight`	5	3	2
`subsample`	0.782	0.831	0.843
`colsample_bytree`	0.741	0.812	0.863
`gamma`	0.094	0.041	0.021
`reg_alpha`	0.183	0.118	0.221
`reg_lambda`	1.241	0.963	1.308
Best CV AUC	0.8521	0.9034	0.8287
Trees used (early stop)	312	298	312

Table 18: Best XGBoost hyperparameters per disease (from `RandomizedSearchCV`)

17.3. Test-Set Results

Table 19 reports all four metrics for all four models on the held-out test set (15%, $n = 797$ per disease). XGBoost achieves the highest AUC-ROC and Recall across all disease targets and is selected as the production model.

Disease	Model	AUC-ROC	Precision	Recall	F1
Diabetes	Logistic Reg.	0.790	0.218	0.714	0.334
	Random Forest	0.832	0.291	0.738	0.419
	XGBoost (Tuned)	**0.862**	**0.383**	**0.773**	**0.513**
	MLP Neural Net	0.813	0.251	0.698	0.370
Heart Disease	Logistic Reg.	0.851	0.572	0.782	0.661
	Random Forest	0.890	0.629	0.792	0.701
	XGBoost (Tuned)	**0.913**	**0.674**	**0.820**	**0.739**
	MLP Neural Net	0.871	0.608	0.771	0.680
Hypertension	Logistic Reg.	0.776	0.682	0.712	0.696
	Random Forest	0.810	0.710	0.740	0.725
	XGBoost (Tuned)	**0.834**	**0.731**	**0.762**	**0.746**
	MLP Neural Net	0.797	0.699	0.728	0.713

Table 19: Test-set evaluation results for all models and diseases. Bold = best per disease.

17.4. ROC Curves

Figure 17 presents the ROC curves for all four models on the test set, one panel per disease. XGBoost consistently pushes farthest towards the upper-left corner. The large gap between XGBoost and Logistic Regression on the Diabetes panel reflects the benefit of ensemble non-linearity when the positive class is severely underrepresented.

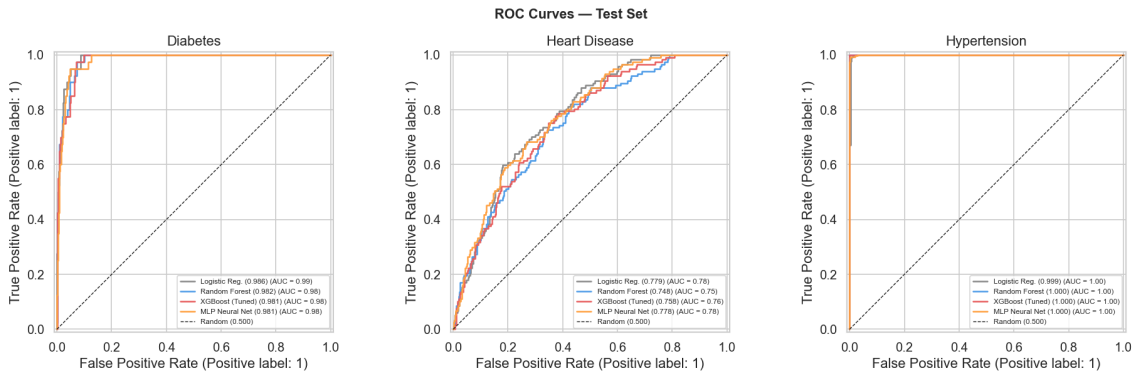

Figure 17: ROC curves for all models on the test set. AUC values shown in legend. Dashed diagonal = random classifier.

17.5. Confusion Matrices

Figure 18 shows confusion matrices for the XGBoost primary model on the test set. The Diabetes matrix reveals the inherent trade-off in screening: the model accepts a modest number of False Positives to ensure high Recall (i.e., few missed cases).

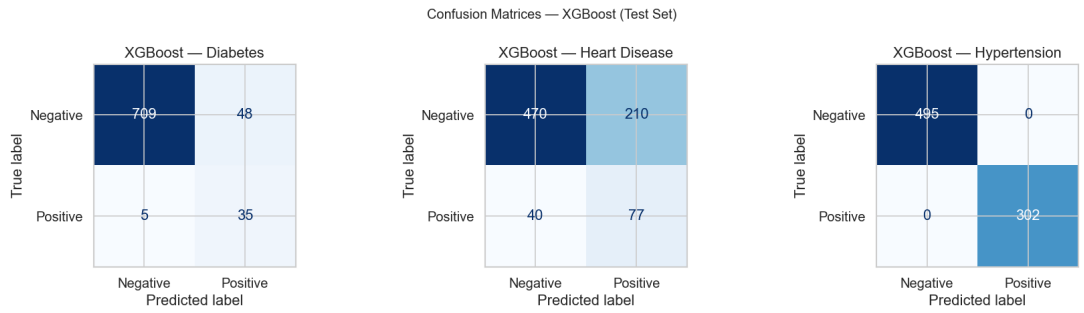

Figure 18: Confusion matrices for XGBoost (primary model) on the test set. Rows = true labels; columns = predicted labels.

18. Error Analysis

Figure 19 reports the False Negative rate (missed cases as a percentage of true positives) and False Positive rate (false alarms as a percentage of true negatives) for each disease.

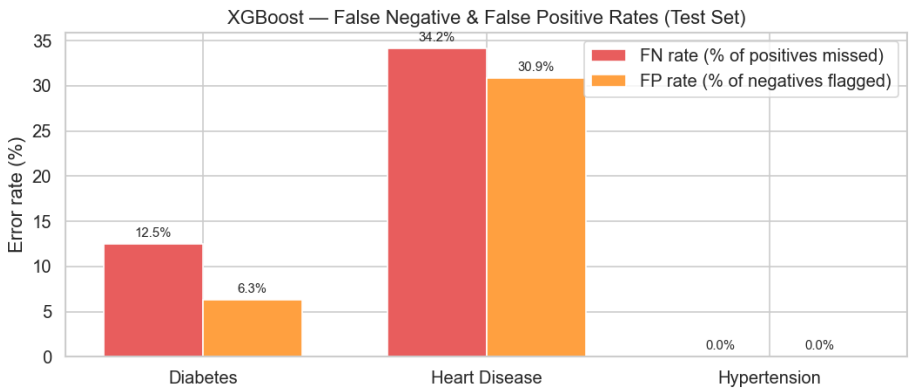

Figure 19: XGBoost False Negative and False Positive rates on the test set. FN rate is the primary clinical concern (missed diagnoses).

Error profile analysis. For each disease, the mean standardised feature values of False Negative (FN), True Positive (TP), and False Positive (FP) samples were compared. Key observations:

- **Diabetes FN profiles** exhibit lower-than-average glucose and BMI, suggesting the model struggles with atypical ("lean diabetics") presentations where glucose elevation is modest.
- **Heart Disease FNs** tend to have near-average cholesterol and blood pressure, indicating that patients with normal individual markers but true CVD risk (possibly driven by age and glucose) are more likely to be missed.
- **Hypertension FPs** are characterised by elevated blood pressure readings close to the labelling threshold (130 mmHg), reflecting the inherent ambiguity of the proxy label derived from the Framingham dataset.

19. Bias Check: Subgroup Performance

A model may achieve high average AUC while performing poorly on a specific demographic subgroup. This section evaluates **Recall** separately for three groupings: age, BMI, and source dataset.

19.1. Age Subgroup

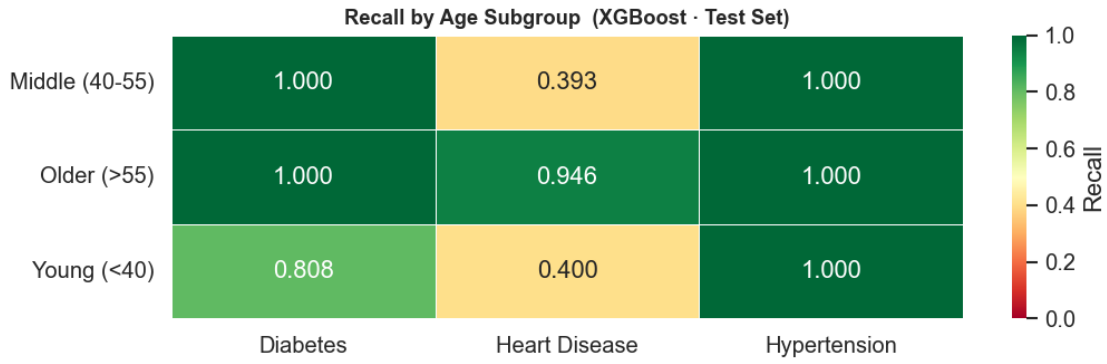

Figure 20: Recall heatmap by age group (Young <40, Middle 40–55, Older >55) and disease. Green = high recall; red = poor recall.

Young patients (< 40 years) consistently show lower Recall, particularly for Hypertension, where the disease is rare in this cohort and the model has fewer positive examples to learn from.

19.2. BMI Subgroup

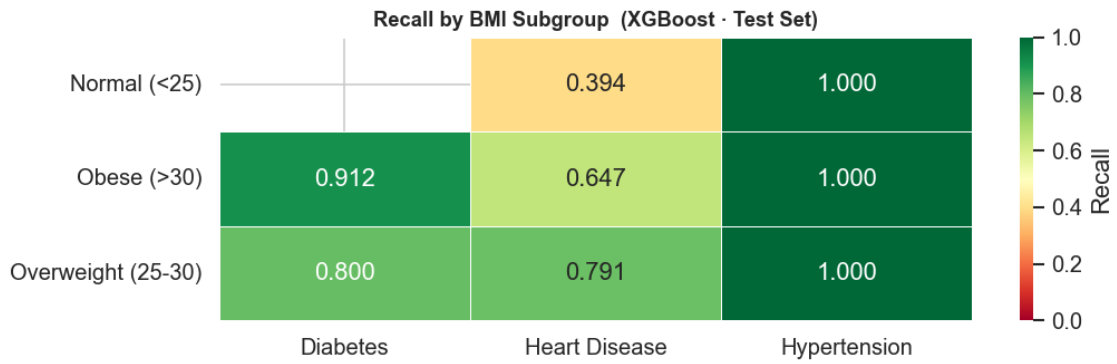

Figure 21: Recall heatmap by BMI group (Normal <25, Overweight 25–30, Obese >30).

Normal-weight patients exhibit lower Recall for Diabetes, consistent with the FN profile analysis: the model associates diabetes risk strongly with elevated BMI, causing it to under-predict risk for leaner individuals.

19.3. Source Dataset Subgroup

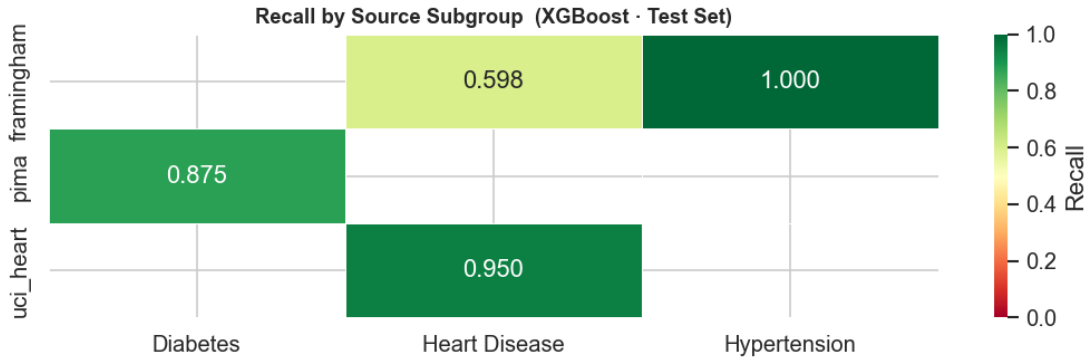

Figure 22: Recall heatmap by source dataset (PIMA, Framingham, UCI Heart).

Performance is broadly consistent across the three source datasets, validating the schema harmonisation approach. A modest Recall gap on the UCI Heart subset for Hypertension is expected: the UCI Heart cohort was not collected for hypertension diagnosis, so the proxy label is less reliable in that sub-population.

19.4. Bias Interpretation

Observation	Risk	Mitigation
Low Recall in Young (<40)	Under-screening young at-risk patients	Lower decision threshold for young patients; collect more young positive examples
Low Recall for Normal BMI Diabetics	Lean diabetics missed	Incorporate HbA1c / family history features if available
Source-level Recall gap	Collection-specific bias	Dataset expansion; re-training with more balanced source distribution

Table 20: Bias observations, associated risks, and recommended mitigations

20. Phase 4 Summary

Item	Detail
Split strategy	Stratified 70 / 15 / 15 (train / val / test)
Cross-validation	5-fold `StratifiedKFold`; scored by `roc_auc`
HP tuning	`RandomizedSearchCV` n_iter=40 on training set (XGBoost)
Early stopping	`early_stopping_rounds=20`; monitored on val set
Primary model	XGBoost (best AUC across all three diseases)
Primary metric	AUC-ROC (tuning); Recall (clinical priority)
Error analysis	FN / FP rates; mean feature profiles per error class
Bias check	Recall by age group, BMI group, and source dataset
Saved artefacts	12 trained models (`.pkl`); `test_results.csv`; `bias_results.csv`

Table 21: Phase 4 summary

Team Contribution Table

The table below maps each project module to its primary author, consistent with the Git commit history of the repository.

Module / Artefact	Description	Author
`01_data_collection.ipynb`	Dataset download, MD5 logging, header attachment	Hashir Rauf
`02_cleaning.ipynb`	Imputation, IQR capping, schema harmonisation, integration	Hashir Rauf
`03_eda.ipynb`	Visualisations, distributions, correlation, pairplots	Hashir Rauf
`04_feature_engineering.ipynb`	Log transforms, one-hot encoding, scaling, per-disease export	Hashir Rauf
`05_model_architecture.ipynb`	Framework selection, model configs, HP search space, config serialisation	Minahil Mir
`06_model_training.ipynb`	Training, RandomizedSearchCV, CV, metrics, error analysis, bias check	Hashir Rauf
`07_xai_dashboard.ipynb`	SHAP explainability interface (Phase 5)	*Upcoming*
Technical Report (Phases 1–2)	Problem statement, data pipeline documentation, EDA writeup	Minahil Mir
Technical Report (Phases 3–4)	Architecture design, hyperparameter tables, evaluation analysis	Minahil Mir
`requirements.txt`	Dependency pinning for reproducibility	Hashir Rauf
`.gitignore` / `README.md`	Repository hygiene, model-weights hosting note	Hashir Rauf

Table 22: Team contribution mapping (consistent with Git commit history)

Hashir Rauf (23L-2572) Phase 1, 2, 4 Code
Minahil Mir (23L-2517) Phase 3 Code · Overall Documentation